Université Bretagne Sud
IUT de Vannes
Département Informatique — 2026/2027

BUT 2 — R3.07. SQL dans un langage de programmation

TD/TP 2

Approfondissement JDBC et bonnes pratiques

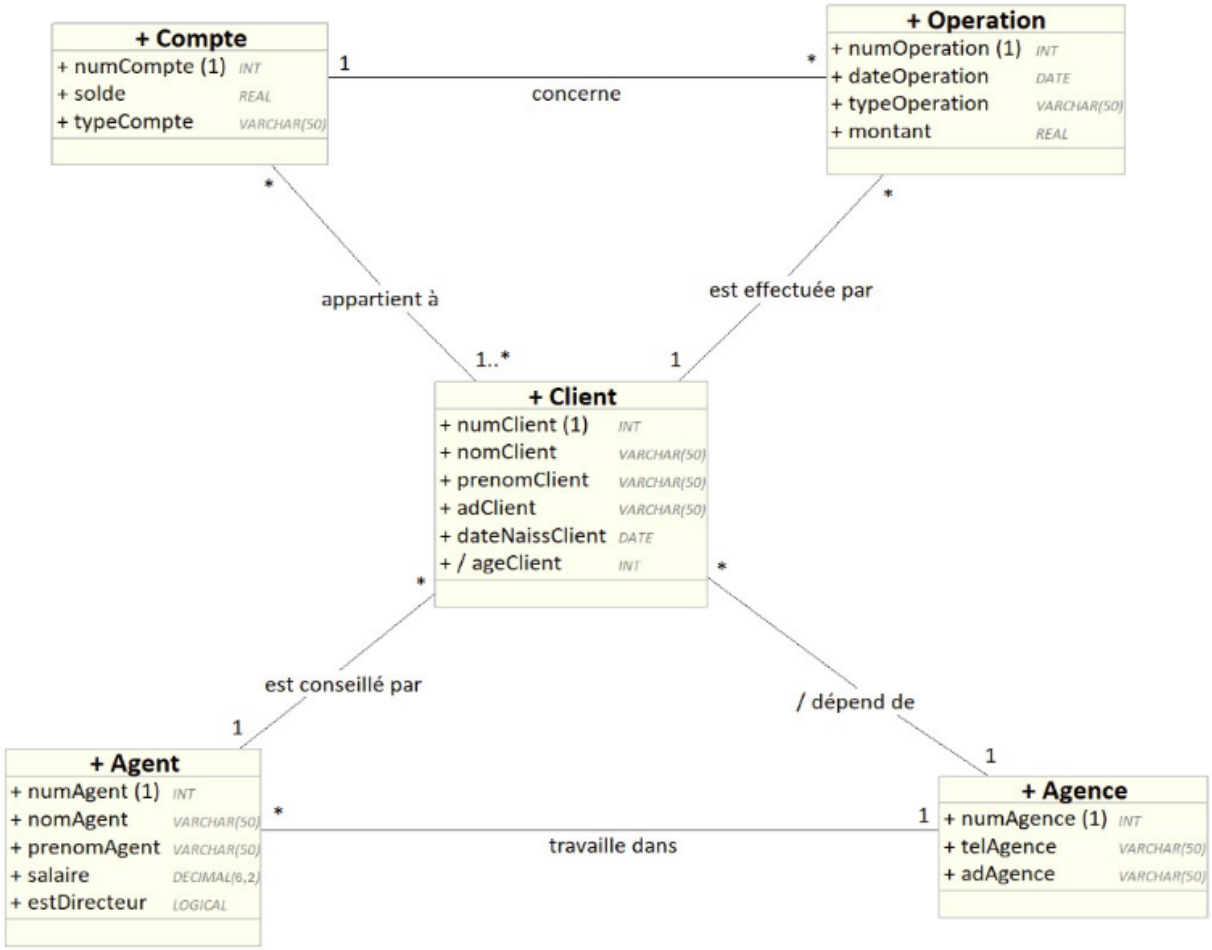
1 Introduction

Dans ce TP, vous allez approfondir l'utilisation de JDBC en ajoutant des contraintes utiles à votre base de données, en sécurisant les informations sensibles avec un fichier `.env`, et en implémentant une classe associative pour modéliser la relation entre `Compte` et `Client`. Vous apprendrez également à réaliser des `INSERT` propres en utilisant des transactions et la gestion d'exceptions.

2 Prérequis

— Avoir terminé le TP précédent (JDBC et Docker).
— Connaître les bases des transactions SQL.
— Savoir utiliser les exceptions en Java.

3 Diagramme de la base de données



Note : Une nouvelle table `Compte_Client` doit être ajoutée pour modéliser la relation entre `Compte` et `Client`.

4 Description des nouvelles contraintes et tables

4.1 Contraintes à ajouter

Vous allez modifier le schéma de la base de données en ajoutant les contraintes suivantes :

- `NOT NULL` sur les colonnes obligatoires.
- `CHECK` sur `ageClient`, `solde`, `montant`, `salaire`.
- `DEFAULT` pour `estDirecteur` (valeur par défaut : `false`).
- `UNIQUE` pour `telAgence`.

Ces contraintes seront ajoutées via des requêtes `ALTER TABLE`, ce qui permet de modifier le schéma sans tout recréer.

4.2 Classe associative `Compte_Client`

Cette table permettra de gérer la relation plusieurs-à-plusieurs entre `Compte` et `Client` :

```
CREATE TABLE IF NOT EXISTS Compte_Client (
    numCompte INT REFERENCES Compte(numCompte),
    numClient INT REFERENCES Client(numClient),
    PRIMARY KEY (numCompte, numClient)
);
```

5 Mise à jour de la base de données

5.1 Étape 1 : Ajouter les contraintes avec `ALTER TABLE`

Modifiez le fichier `init.sql` pour ajouter les contraintes après la création des tables :

```sql
-- Exemple pour la table Client
ALTER TABLE Client
    ALTER COLUMN nomClient SET NOT NULL,
    ALTER COLUMN prenomClient SET NOT NULL,
    ADD CONSTRAINT chk_age CHECK (ageClient >= 18);

-- Exemple pour la table Compte
ALTER TABLE Compte
    ADD CONSTRAINT chk_solde CHECK (solde >= 0);

-- Exemple pour la table Agence
ALTER TABLE Agence
    ADD CONSTRAINT unique_telAgence UNIQUE (telAgence);

-- Exemple pour la table Agent
ALTER TABLE Agent
    ALTER COLUMN estDirecteur SET DEFAULT false;
```

Grace à cette écriture différentielle on modifie uniquement ce qui change, ce qui est plus maintenable et moins risqué.

5.2 Étape 2 : Créer un fichier `.env`

Pour sécuriser les informations sensibles, créez un fichier `.env` à la racine de votre projet, qu'il ne faudra **jamais versionner** :

```
DB_URL=jdbc:postgresql://db:5432/banque_db
DB_USER=user
DB_PASSWORD=password
```

D'abord modifier le docker-compose.yml pour intégrer le fichier .env à la place des variables d'environnement déclarées :

1. supprimer les lignes suivantes :

```
environment:
      DB_URL: jdbc:postgresql://db:5432/banque_db
      DB_USER: user
      DB_PASSWORD: password
```

2. ajouter la ligne `env_file` comme suit (le faire pour les 2 services) :

```
services:
  app:
    build: .
    depends_on:
      - db
    ports:
      - "8080:8080"
    env_file:
      - .env
```

Les variables définies dans `.env` (comme `DB_USER`, `DB_PASSWORD`, etc.) seront automatiquement chargées par Docker. Assurez-vous que les noms des variables dans `.env` correspondent à ceux attendus par votre application (par exemple, `POSTGRES_USER` pour le service `db`).

Dans votre application, utilisez une bibliothèque comme `dotenv-java` pour charger ces variables. De cette manière, on ne les définit qu'à un seul endroit, *i.e.* dans le fichier `.env`

6 Insertions propres avec transactions et gestion d'exceptions

6.1 Exemple de code Java

Voici comment réaliser un `INSERT` propre :

```java
public void ajouterClient(Client client) throws SQLException {
    String sql = "INSERT INTO Client (numClient, nomClient, prenomClient, adClient,
    dateNaissClient, ageClient, numAgent) VALUES (?, ?, ?, ?, ?, ?, ?)";

    try (Connection conn = getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql)) {
        // Verification manuelle avant insertion
        if (clientExisteDeja(client.getNumClient())) {
            throw new SQLException("Client deja existant !");
        }

        // Debut de la transaction
        conn.setAutoCommit(false);

        pstmt.setInt(1, client.getNumClient());
        pstmt.setString(2, client.getNomClient());
        pstmt.setString(3, client.getPrenomClient());
        pstmt.setString(4, client.getAdClient());
        pstmt.setDate(5, client.getDateNaissClient());
        pstmt.setInt(6, client.getAgeClient());
        pstmt.setInt(7, client.getNumAgent());

        pstmt.executeUpdate();
        conn.commit(); // Validation de la transaction

    } catch (SQLException e) {
        conn.rollback(); // Annulation en cas d'erreur
        throw e;
    }
}

private boolean clientExisteDeja(int numClient) throws SQLException {
    String sql = "SELECT 1 FROM Client WHERE numClient = ?";
    try (PreparedStatement pstmt = getConnection().prepareStatement(sql)) {
        pstmt.setInt(1, numClient);
        return pstmt.executeQuery().next();
    }
}
```

Pour faire les choses encore plus proprement, on ne se contente pas de traiter les requêtes de type `SQLException`. On Distingue les erreurs selon leur type (ex : violation de contrainte, connexion perdue). Par exemple :

```java
try {
    // ...
} catch (SQLIntegrityConstraintViolationException e) {
    System.err.println("Violation de contrainte : " + e.getMessage());
} catch (SQLException e) {
    System.err.println("Erreur SQL : " + e.getMessage());
}
```

7 Tâches à réaliser

7.1 Méthodes à implémenter

Ajoutez les méthodes suivantes dans `BanqueDAO` :

— `ajouterCompteClient(int numCompte, int numClient)` : pour gérer la relation entre un compte et un client.
— `obtenirComptesParClient(int numClient)` : pour récupérer tous les comptes d'un client.

7.2 Classes métier mises à jour

Ajoutez la classe `CompteClient` :

```
public class CompteClient {
    private int numCompte;
    private int numClient;
    // Constructeurs, getters et setters
}
```

7.3 Conseils

— Testez chaque contrainte avec des requêtes SQL avant de les ajouter.
— Utilisez des `PreparedStatement` pour éviter les injections SQL.
— Validez vos transactions uniquement après avoir vérifié les données.

8 Concurrence d'accès (3e séance)

Cette partie met en pratique le chapitre 2. Deux guichets débitent **le même compte au même moment**. On part d'un compte `numCompte = 1` dont le solde vaut 1000 (adaptez le numéro à vos données).

8.1 Observer l'anomalie dans deux terminaux `psql`

Exercice 1 — Mise à jour perdue « à la main »

Ouvrez **deux** terminaux, et dans chacun un shell `psql` sur la base (`docker exec -it <conteneur> psql -U user -d banque_db`). Exécutez les commandes **dans l'ordre indiqué**, en alternant les terminaux :

Étape	Terminal A	Terminal B
1	`BEGIN;`	
2	`SELECT solde FROM Compte WHERE numCompte = 1;`	
3		`BEGIN;`
4		`SELECT solde FROM Compte WHERE num`
5		`UPDATE Compte SET solde = 950 WHER`
6		`COMMIT;`
7	`UPDATE Compte SET solde = 900 WHERE numCompte = 1;`	
8	`COMMIT;`	

a) Quel est le solde final ? Quel aurait dû être le solde après deux débits de 100 et 50 ?
b) Remettez le solde à 1000 et recommencez en remplaçant les deux `BEGIN;` par `BEGIN`

`ISOLATION LEVEL REPEATABLE READ;`. Que se passe-t-il à l'étape 7 ? Notez le message d'erreur.

c) Recommencez au niveau par défaut, mais avec `SELECT ... FOR UPDATE` aux étapes 2 et 4. Que se passe-t-il à l'étape 4 ? Passez alors directement aux étapes 7 et 8 dans le terminal A. Quelle valeur le terminal B affiche-t-il ensuite ? Terminez B en écrivant le solde correct (étapes 5 et 6).

8.2 Reproduire l'anomalie en Java

Ajoutez à `BanqueDAO` une méthode volontairement **naïve** qui lit le solde, attend un peu (pour élargir la fenêtre de concurrence), puis écrit le nouveau solde calculé en Java :

```
public void debiterNaif(Connection c, int numCompte, double montant)
    throws SQLException, InterruptedException {
  double solde;
  try (PreparedStatement ps = c.prepareStatement(
        "SELECT solde FROM Compte WHERE numCompte = ?")) {
    ps.setInt(1, numCompte);
    ResultSet rs = ps.executeQuery();
    rs.next();
    solde = rs.getDouble("solde");
  }
  Thread.sleep(200);   // simule un traitement metier
  try (PreparedStatement ps = c.prepareStatement(
        "UPDATE Compte SET solde = ? WHERE numCompte = ?")) {
    ps.setDouble(1, solde - montant);
    ps.setInt(2, numCompte);
    ps.executeUpdate();
  }
}
```

Puis lancez deux guichets en parallèle, chacun avec **sa propre connexion** :

```
ExecutorService pool = Executors.newFixedThreadPool(2);
for (double montant : new double[] {100, 50}) {
  pool.submit(() -> {
    try (Connection c = BanqueDAO.getConnection()) {
      c.setAutoCommit(false);
      new BanqueDAO(c).debiterNaif(c, 1, montant);
      c.commit();
    } catch (Exception e) {
      System.err.println("Echec : " + e.getMessage());
    }
  });
}
pool.shutdown();
pool.awaitTermination(10, TimeUnit.SECONDS);
// afficher ensuite le solde du compte 1
```

Exercice 2 — Corriger la mise à jour perdue

a) Exécutez le programme plusieurs fois (en remettant le solde à 1000). Constatez la mise à jour perdue.

b) Écrivez `debiter(Connection c, int numCompte, double montant)` avec une **seule** requête `UPDATE Compte SET solde = solde - ? WHERE numCompte = ? AND solde >= ?`. Si aucune ligne n'est modifiée, levez une exception métier `SoldeInsuffisantException`. Vérifiez que le solde final est correct.

c) Écrivez une variante `debiterAvecVerrou` qui garde la lecture en Java mais utilise `SELECT`

... `FOR UPDATE`. Vérifiez le résultat et mesurez la durée totale : pourquoi est-elle plus longue ?

d) **Pour aller plus loin :** gardez `debiterNaif` mais passez la transaction en `Connection.TRANSACTION_REPEATABLE_READ`. Affichez le `getSQLState()` de l'exception reçue par l'un des deux guichets, puis ajoutez une boucle qui **rejoue** la transaction (au plus 3 fois) quand le code vaut `40001`.

e) Dans votre README, expliquez en quelques lignes quelle solution vous retenez pour un virement, et pourquoi.

9 Exercice supplémentaire (facultatif) : logs avec Log4j2

Pour améliorer le débogage et la maintenance de votre application, vous allez intégrer la bibliothèque **Log4j2** pour tracer les opérations critiques (connexions, requêtes, erreurs).

9.1 Étape 1 : Ajouter la dépendance Maven

Modifiez votre fichier `pom.xml` pour inclure Log4j2 :

```xml
<dependencies>
    <!-- Dependance existante pour JDBC/PostgreSQL -->
    <dependency>
        <groupId>org.postgresql</groupId>
        <artifactId>postgresql</artifactId>
        <version>42.6.0</version>
    </dependency>

    <!-- Ajoutez Log4j2 -->
    <dependency>
        <groupId>org.apache.logging.log4j</groupId>
        <artifactId>log4j-core</artifactId>
        <version>2.20.0</version>
    </dependency>
    <dependency>
        <groupId>org.apache.logging.log4j</groupId>
        <artifactId>log4j-api</artifactId>
        <version>2.20.0</version>
    </dependency>
</dependencies>
```

9.2 Étape 2 : Configurer Log4j2

Créez un fichier `log4j2.xml` dans `src/main/resources` :

```xml
<?xml version="1.0" encoding="UTF-8"?>
<Configuration status="WARN">
    <Appenders>
        <Console name="Console" target="SYSTEM_OUT">
            <PatternLayout pattern="%d{HH:mm:ss.SSS} [%t] %-5level %logger{36} - %msg%n"/>
        </Console>
    </Appenders>
    <Loggers>
        <Root level="info">
            <AppenderRef ref="Console"/>
        </Root>
        <Logger name="com.votreprojet.dao" level="debug" additivity="false">
            <AppenderRef ref="Console"/>
        </Logger>
```

```
    </Loggers>
</Configuration>
```

9.3 Étape 3 : Instrumenter la classe `BanqueDAO`

Modifiez la classe `BanqueDAO` pour ajouter des logs aux méthodes existantes :

```
import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;

public class BanqueDAO {
    private static final Logger logger = LogManager.getLogger(BanqueDAO.class);
    private Connection connection;

    public BanqueDAO(Connection connection) {
        this.connection = connection;
        logger.info("Initialisation de BanqueDAO avec une nouvelle connexion");
    }

    public void ajouterClient(Client client) throws SQLException {
        logger.debug("Tentative d'ajout du client : {}", client.getNomClient());

        String sql = "INSERT INTO Client (numClient, nomClient, prenomClient, adClient,
    dateNaissClient, ageClient, numAgent) VALUES (?, ?, ?, ?, ?, ?, ?)";

        try (PreparedStatement pstmt = connection.prepareStatement(sql)) {
            pstmt.setInt(1, client.getNumClient());
            pstmt.setString(2, client.getNomClient());
            pstmt.setString(3, client.getPrenomClient());
            pstmt.setString(4, client.getAdClient());
            pstmt.setDate(5, client.getDateNaissClient());
            pstmt.setInt(6, client.getAgeClient());
            pstmt.setInt(7, client.getNumAgent());

            int rowsAffected = pstmt.executeUpdate();
            logger.info("Client {} ajoute avec succes. Lignes affectees : {}", client.
    getNomClient(), rowsAffected);

        } catch (SQLException e) {
            logger.error("Erreur lors de l'ajout du client {} : {}", client.getNomClient(), e.
    getMessage());
            throw e;
        }
    }

    public Client obtenirClientParNum(int numClient) throws SQLException {
        logger.debug("Recherche du client avec numClient = {}", numClient);

        String sql = "SELECT * FROM Client WHERE numClient = ?";
        try (PreparedStatement pstmt = connection.prepareStatement(sql)) {
            pstmt.setInt(1, numClient);
            ResultSet rs = pstmt.executeQuery();

            if (rs.next()) {
                Client client = new Client(
                    rs.getInt("numClient"),
                    rs.getString("nomClient"),
                    rs.getString("prenomClient"),
                    rs.getString("adClient"),
                    rs.getDate("dateNaissClient"),
                    rs.getInt("ageClient"),
                    rs.getInt("numAgent")
                );
```

```
                logger.info("Client trouve : {}", client.getNomClient());
                return client;
            } else {
                logger.warn("Aucun client trouve avec numClient = {}", numClient);
                return null;
            }
        }
    }
}
```

9.4 Étape 4 : Tester les logs

— Exécutez votre application et observez les logs dans la console.
— Vérifiez que les messages de niveau `INFO` (ajout réussi) et `ERROR` (erreur) apparaissent correctement.
— Forcez une erreur (ex : violation de contrainte) et vérifiez que le log d'erreur s'affiche.

9.5 Questions

1. Quel est l'intérêt d'utiliser des niveaux de log différents (`DEBUG`, `INFO`, `ERROR`) ?
2. Comment modifieriez-vous la configuration pour écrire les logs dans un fichier `app.log` en plus de la console ?
3. Ajoutez des logs dans la méthode `ajouterOperation` pour tracer le montant et le type d'opération.

Indice : Pour écrire dans un fichier, ajoutez un `FileAppender` dans `log4j2.xml` :

```
<Appenders>
    <File name="File" fileName="logs/app.log">
        <PatternLayout pattern="%d{HH:mm:ss.SSS} [%t] %-5level %logger{36} - %msg%n"/>
    </File>
</Appenders>
```